

Enhance Your Application's Performance with AMQP and Log-Based Systems

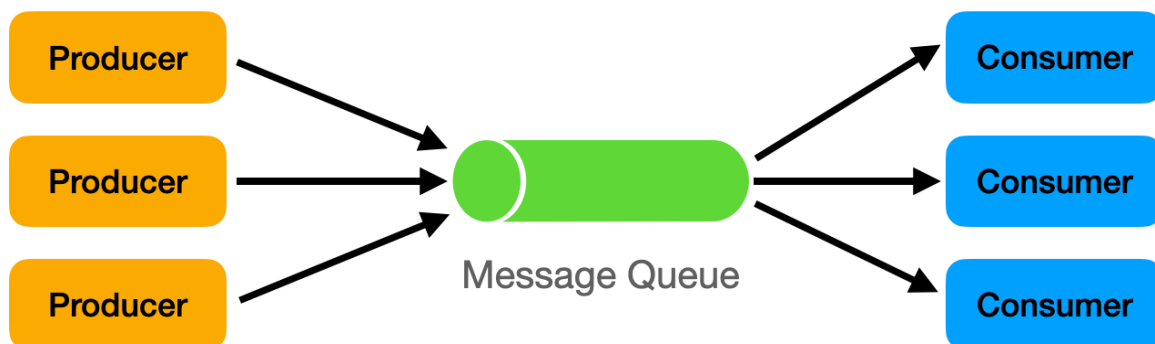
 systemdesignschool.io/fundamentals/message-queue



Message Queues in System Design

A message queue (also called message broker) is a software component that allows communication between different applications or subsystems of an application by passing messages. An analogy is that it's like emails, not between people but rather between app services.

The messages are usually small, and can be things like requests, replies, error messages, or just plain information. To send a message, a component called a producer adds a message to the queue. To receive a message, a component called a consumer retrieves the message, processes the message, and acknowledges that the work is done by removing the message from the queue. The queue is a buffer between producers and consumers.



Main advantages of using message queues:

1. Acts as a buffer between services when the producer and consumer have different processing rates.

Message queues can help manage the difference in processing rates between the producer and the consumer. For example, services that involve writing to a database are generally slower than stateless web services that process requests in memory. By using a message queue, the producer can send messages at its own pace without overwhelming the slower consumer.

Example:

- In a photo-sharing application, users may upload photos rapidly, and the producer sends the photos to the message queue quickly. However, the photo uploader and database updater services might not be able to process the images at the same speed. Using a message queue allows the consumer services to process the images at their own pace without losing any data.
 - On a video-sharing platform like YouTube, users may post comments at a high rate. The front-end service can handle posting comments quickly, but the database updater service responsible for storing the comments might not keep up with the same speed. Using a message queue allows the database updater to process the comments at its own pace without losing any data.
2. Decouples components (producer and consumer). If the consumer goes down, the message is still in the queue and can be processed when it comes back up.
 - Message queues provide a level of fault tolerance by decoupling the producer and consumer components. If a consumer service goes down or experiences an issue, the messages remain in the queue, ready to be processed when the consumer comes back online.
 - In the previous examples, if either photo uploader or database updater goes down, the message is still in the queue and can be processed when the service recovers.

Two types of message queues

AMQP/JMS-style message queue

These message brokers are based on the Advanced Message Queuing Protocol (AMQP) or Java Message Service (JMS) standards. These are originally designed to pass messages in large enterprise systems. They are designed to support various messaging patterns like publish/subscribe, point-to-point, and request/reply.

- Messages are typically held in memory, and once they are consumed by the intended recipient, they are deleted. These brokers emphasize message routing, filtering, and transformation.
- There could be duplicate messages and ordering is either not guaranteed or per-queue guaranteed (no global ordering).

Examples: RabbitMQ, ActiveMQ, Azure Service Bus, IBM MQ

Log-based message queue

These message queues, such as Kafka or AWS Kinesis, use a distributed, append-only log for storing and processing messages. The broker assigns all messages in a partition to the same consumer node, and always delivers messages in the same order. Parallelism is achieved through partitioning, and consumers track their progress by checkpointing the offset of the last message they have processed. The broker retains messages on disk, so it is possible to jump back and reread old messages if necessary.

Note that log-based message queues are often referred to as message **streaming** platforms. Kafka's official definition is "an open-source distributed **event streaming platform**". The main difference between a stream and a queue is that messages sent to a stream are persisted and can be read back using an iterator, while messages sent to a queue are processed independently and deleted when done. We will discuss more about stream processing in a later chapter.

- Messages are persisted on disk, allowing for long-term storage, reprocessing, and increased fault tolerance.
- They focus on high-throughput, low-latency processing, and are optimized for data streaming and real-time analytics.
- Log-based brokers often provide strong durability guarantees and can handle high volumes of data.
- Messages within partitions are totally ordered, and each message has a unique offset for maintaining the order.

Examples: Apache Kafka, AWS Kinesis, Google Cloud Pub/Sub, Azure Event Hubs

Note that just like database technologies, the line gets blurry as one type of queues adds features of the other. For example, IBM MQ [added message deduplication](#) after v9.2.3. Amazon SQS is a hybrid of the two styles. It's queue centric, has message acknowledgement and deletion but also has message retention and distributed.

Finally, here is a table comparing the two styles of message queues.

	AMQP/JMS-style Traditional Message Queue	Log-based Message Queue
Message Storage	In-memory (usually transient)	Persistent (on-disk storage)
Message Durability	Optional (configurable)	Strong durability guarantees
Message Ordering	Unordered or per-queue ordering	Partition-based ordering

	AMQP/JMS-style Traditional Message Queue	Log-based Message Queue
When to use	Messages are expensive to process, order not important	Messages are fast to process, ordering is important
Example use case	Encoding videos, fetching hotel prices, downloading large files	Analytics stream processing, event sourcing (recording video game moves, payment system processing), IOT sensor data